UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Faculty of Artificial Intelligence

TOPIC 8:
INTERACTIVE DATA VISUALIZATION WITH PLOTLY

DATA WRANGLING AND VISUALIZATION (SAIA2143)

Prepared by:
Ts. Dr. Norziha Megat Mohd Zainuddin
norziha.kl@utm.my

COURSE LEARNING OUTCOMES

At the end of this topic, students should be able to:

a. Understand the role and benefits of interactive visualizations.
b. Use Plotly to create various interactive charts and dashboards.
c. Analyze datasets using interactive visual exploration.
d. Apply learned techniques to a hands-on case study.

TOPIC 8: INTERACTIVE DATA VISUALIZATION WITH PLOTLY

a. Introduction to interactive visualizations

b. Creating dynamic and interactive plots with Plotly

c. Case studies in interactive data exploration

Static vs Interactive Visualizations

Static

- Rendered as images (e.g., PNG, PDF)
- No user interaction
- **Tools:** Matplotlib, Seaborn





Interactive

- Allow zoom, hover, filter, animation
- Browser-based, real-time updates
- **Tools:** Plotly, Bokeh, Altair, D3.js









CLASS ACTIVITY



Instruction:

1. Where have you seen interactive visualizations in real life?
2. What makes a visualization engaging?
3. Why do organizations use interactive visuals?

Why Interactive Visuals?



BETTER DATA STORYTELLING



ENHANCED USER ENGAGEMENT

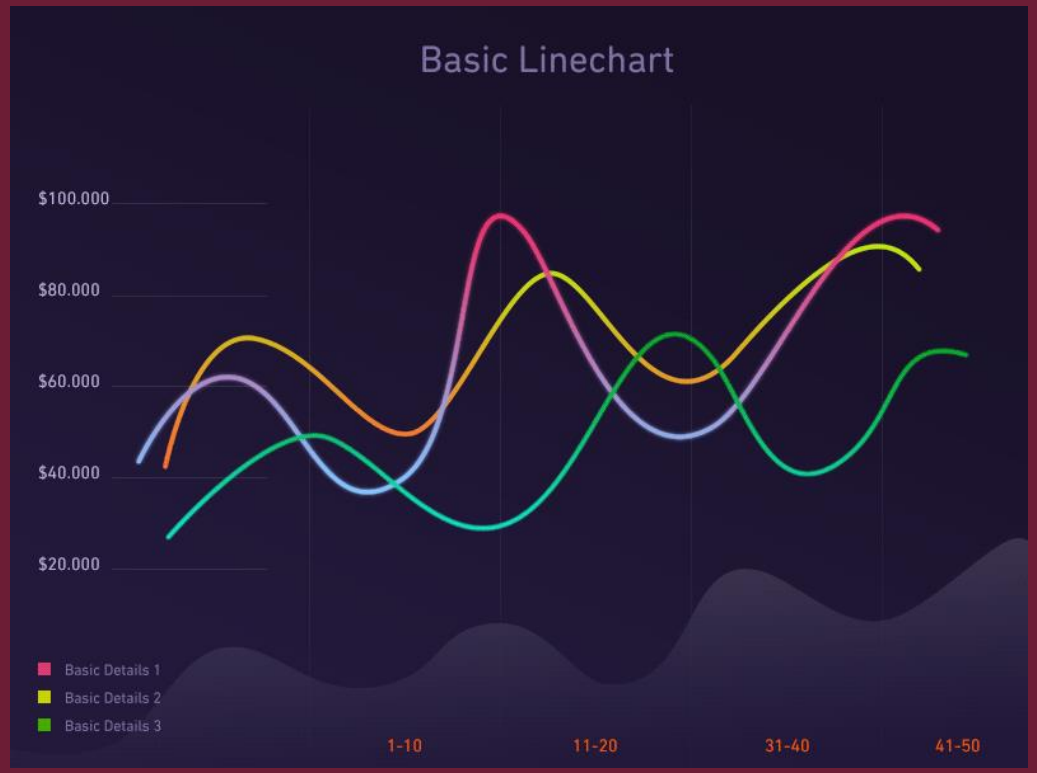


REAL-TIME INSIGHTS EXPLORATION

Why animate?

- The animation should add value, such as
  a. Showing trends over time,
  b. Transitions
  c. Comparisons

- Common goals:
  a. Show change over time
  b. Highlight differences between categories
  c. Display cumulative or progressive data



TOPIC 8: INTERACTIVE DATA VISUALIZATION WITH PLOTLY

a. Introduction to interactive visualizations

b. Creating dynamic and interactive plots with Plotly

c. Case studies in interactive data exploration

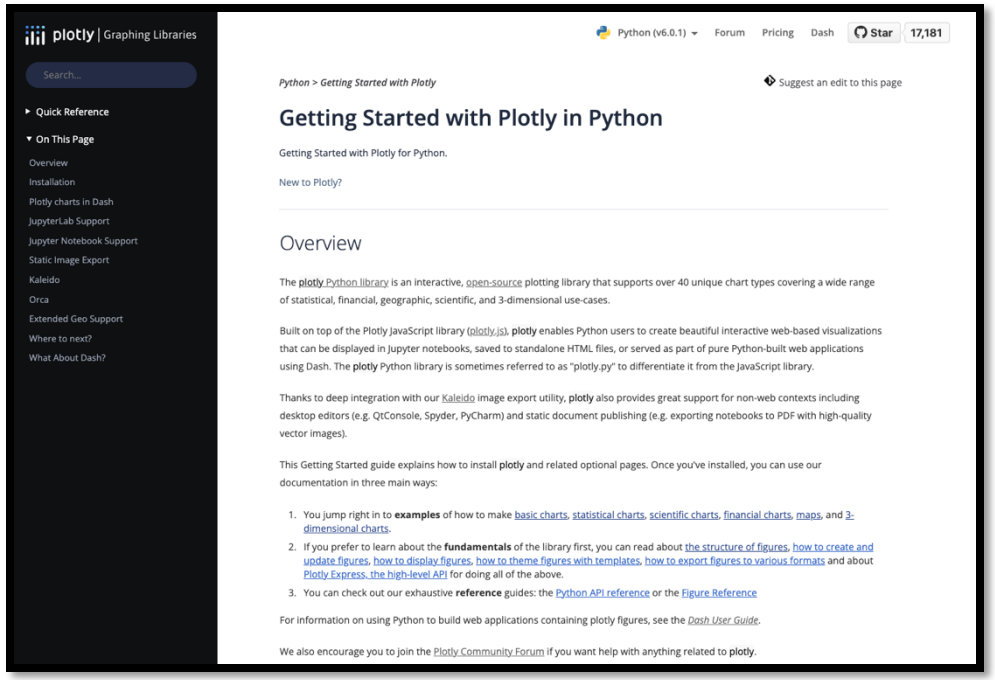
What is plotly

- Open-source graphing library for Python, R, JavaScript
- Used for creating interactive, web-ready visualizations
- Key Features:
  a. Works in Jupyter, Colab, and web apps (Dash)
  b. Two main interface: high-level and low-level of API
- Output modes:
  i. Static HTML (offline)
  ii. Embeddable in Jupyter or websites
  iii. Extendable to Dash apps for interactivity

Why Use

- Pure Python, but renders in HTML
- Advantages Over Other Libraries:
  a. More interactive than Seaborn/Matplotlib
  b. More intuitive than Bokeh for beginners
  c. Smooth integration with Jupyter, Dash, Flask

CLASS ACTIVITY



Instruction:

Please visit this web about Plotly

https://plotly.com/python

Example

The **Iris dataset** is a classic and widely used dataset in data science and machine learning. It was first introduced by the British biologist **Ronald A. Fisher** in 1936.

Purpose: Used to demonstrate classification techniques and visualizations.

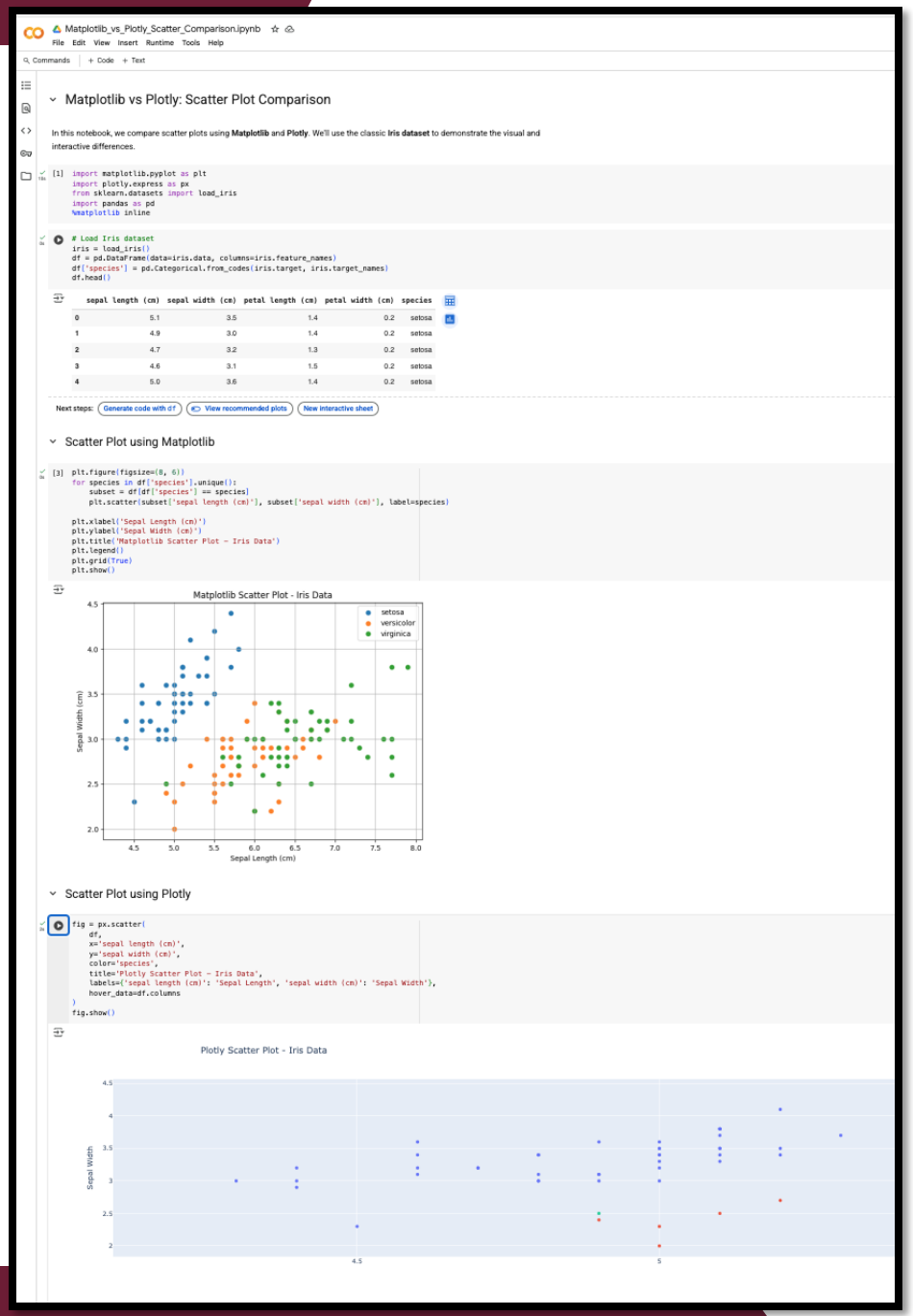
Contents: Measurements of 150 iris flowers from **3 species**:

- *Iris Setosa*
- *Iris Versicolor*
- *Iris Virginica*









Instruction:

1. Compare: Matplotlib scatter vs Plotly scatter using **Fisher's *Iris*** data set using Google Colab.
2. How about HTML files? Iris_Interactive_Visualization.html

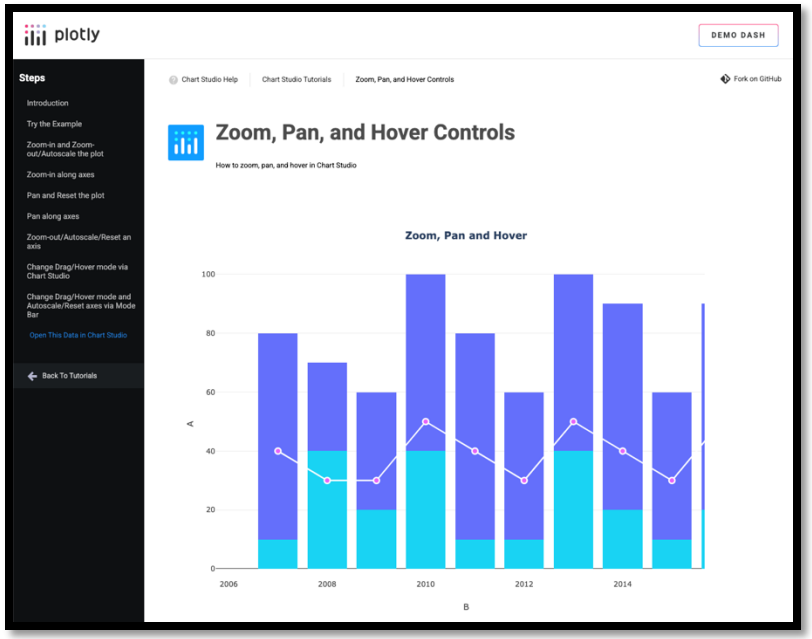


Google Colab file:

Matplotlib_vs_Plotly_Scatter_Comparison.ipynb

www.fai.utm.my

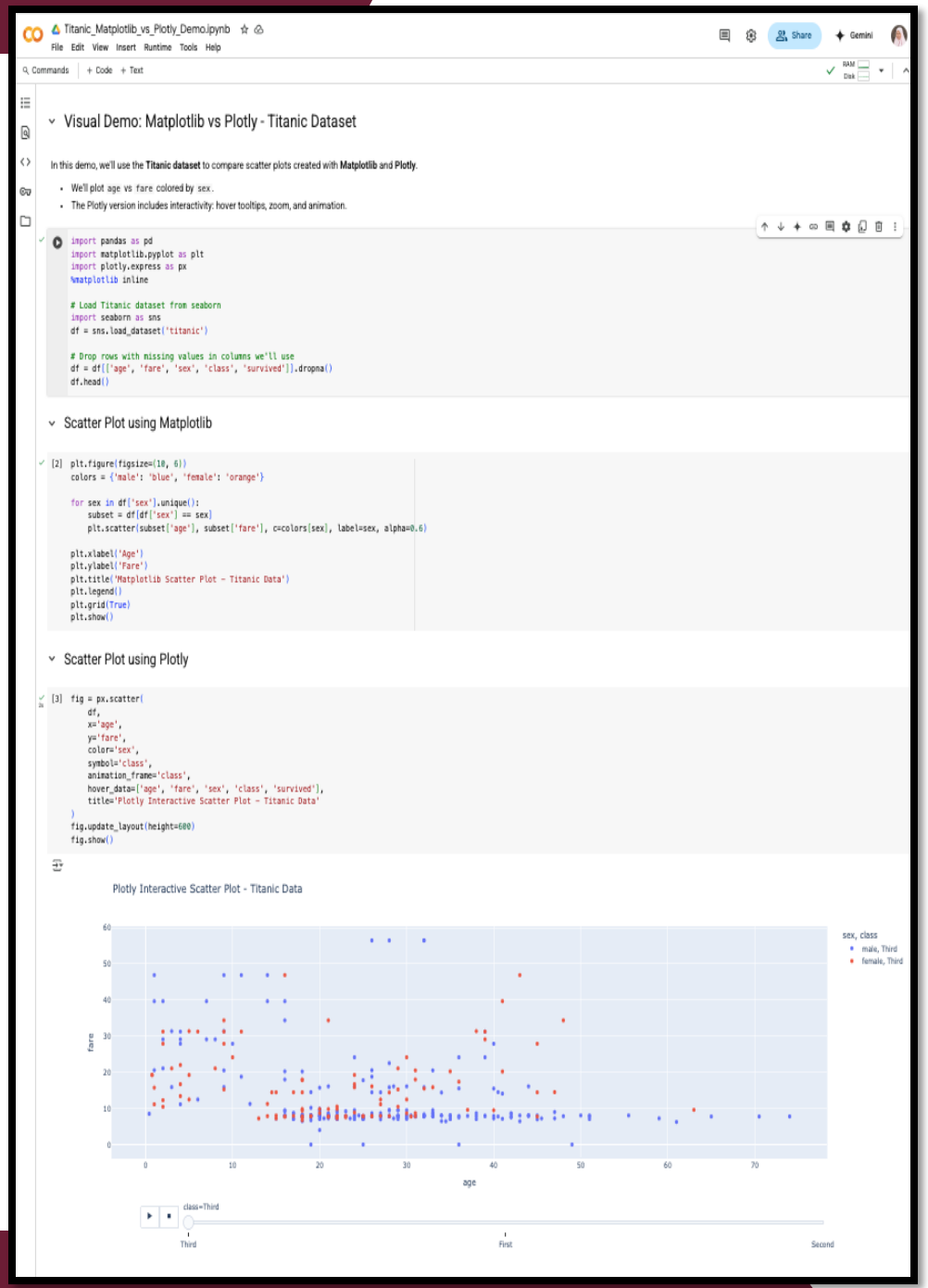
CLASS ACTIVITY



Instruction:

Using Plotly's built-in interactivity tools (DEMO DASH) such as zoom, pan, hover, and autoscale by exploring and modifying visualizations.

https://plotly.com/chart-studio-help/zoom-pan-hover-controls/#step-3-zoom-in-and-zoom-out-autoscale-the-plot



Instruction:

1. Which part of Plotly is interactivity?
2. Identify what is the meaning in plotly:
   i. Hover tooltips
   ii. Zoom
   iii. Animation
   iv. Pan

Google Colab file:

Titanic_Matplotlib_vs_Plotly_Demo.ipynb

Installing Plotly in Python

- Plotly can be used in Jupyter, Colab, and web applications.
- Install Plotly with pip:

```
pip install plotly
```

- No need to install if using Google Colab (already preinstalled).

High-Level vs Low-Level Plotly API

Feature	High-Level API (`plotly.express`)	Low-Level API (`plotly.graph_objects`)
Purpose	Quick and easy plotting	Full control and deep customization
Lines of Code	Very few	Usually more
Best For	Fast prototyping, common charts	Complex, customized visualizations
Defaults	Many good defaults (auto-labels, color maps)	Must define many things manually
Flexibility	Limited (but still powerful)	Maximum flexibility and customization
Chart Types	Basic (scatter, bar, line, etc.)	All chart types including 3D, subplots, custom layouts

Basic Syntax

High-Level API:

```
# High-level bar chart using plotly.express
fig = px.bar(df, x='City', y='Population', title='City Population - High-Level API')
fig.show()
```

Low-Level API:

```
# Low-level bar chart using plotly.graph_objects
fig = go.Figure(data=[
    go.Bar(name='Population', x=df['City'], y=df['Population'])
])
fig.update_layout(title='City Population - Low-Level API',
                  xaxis_title='City',
                  yaxis_title='Population')
fig.show()
```



Instruction:

Identify the difference on the coding for the bar chat using:

a. High-level bar chart - **`plotly.express`**

b. Low-level bar chart - **`plotly.graph_objects`**

Google Colab file:

Plotly_API_Comparison.ipynb

www.fai.utm.my

Example of Output Difference

High-level bar chart -
`plotly.express`

- Automatically adds a legend and colors
- Axis labels and title are handled for you
- Hover tool shows data nicely formatted
- You get a polished interactive chart *fast*

Low-level bar chart -
`plotly.graph_objects`

- You must define marker color, title, axis labels
- More code, but also more control (e.g., marker size, shape, style)
- Essential when combining plots or doing multi-panel plots

Adding Interactivity - Tooltips and Hover

Code Example:

```
fig = px.scatter(
    df, x='x', y='y', color='label',
    hover_data=['feature1', 'feature2']
)
fig.show()
```

Notes:

- Hover data adds extra info
- Useful for Exploratory Data Analysis (EDA) and dashboards

Adding Interactivity - Zoom and Pan

Code Example:

```
fig.update_layout(dragmode='zoom')
```

How It Works:

- Default behavior in all Plotly charts
- Use mouse scroll or box select
- Reset with toolbar icon

Adding Animation and Sliders

Code Example:

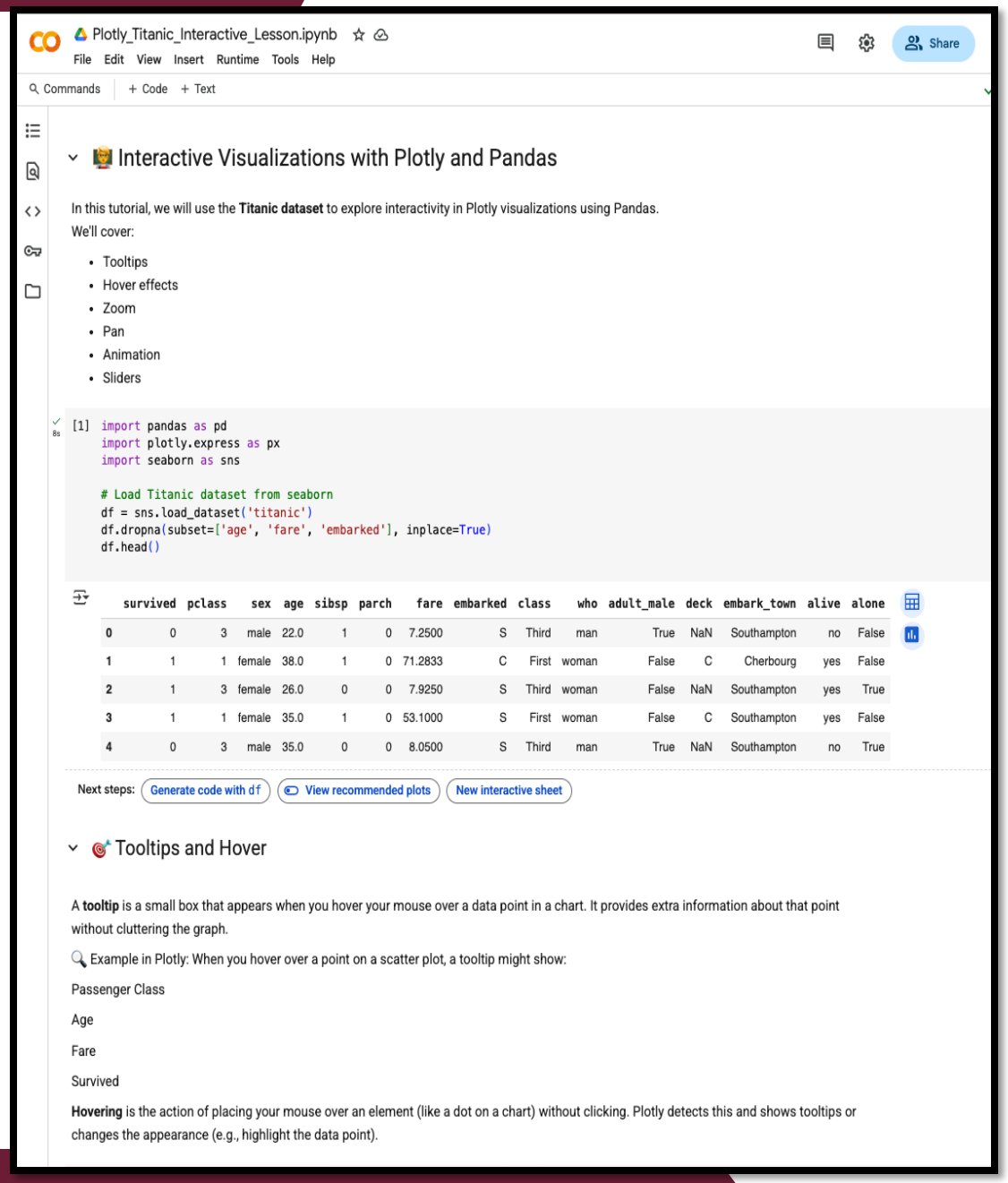
```
fig = px.scatter(
    df, x='gdpPercap', y='lifeExp',
    animation_frame='year', animation_group='country',
    size='pop', color='continent', hover_name='country'
)
fig.show()
```

Use Cases:

- Trend over time
- Gapminder-style visuals

Key Parameters in Plotly Express Animation

Parameter	Purpose
`animation_frame`	The column that controls frame changes
`animation_group`	Groups related points together across frames
`range_x, range_y`	Fix axes so they don't re-scale every frame



Please explore:

- Tooltips and hover interactions
- Zoom and pan functionality
- Animation based on embarkation port
- Sliders for controlling histogram bins

Google Colab file:

Plotly_Titanic_Interactive_Lesson.ipynb

TOPIC 8: INTERACTIVE DATA VISUALIZATION WITH PLOTLY

a. Introduction to interactive visualizations

b. Creating dynamic and interactive plots with Plotly

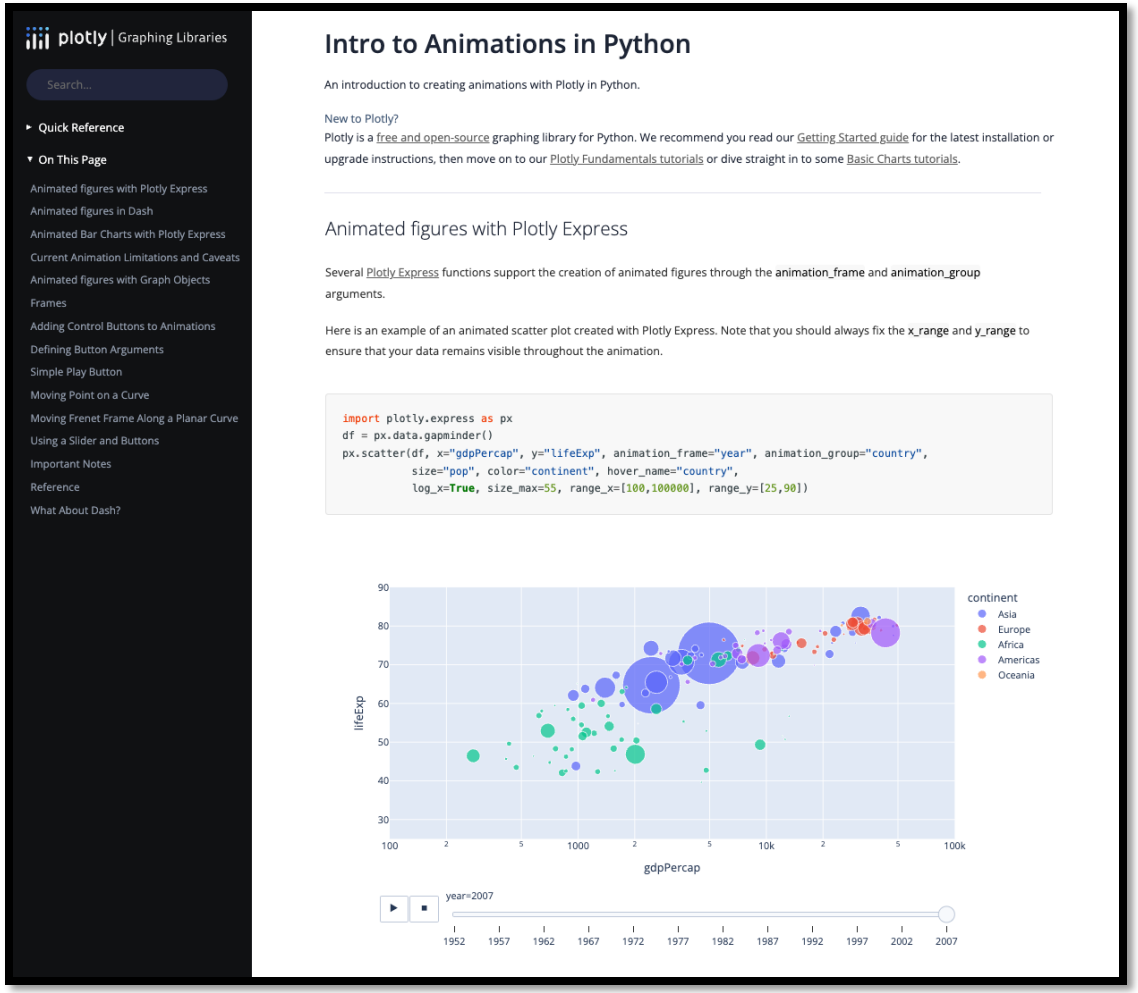
c. Case studies in interactive data exploration

CLASS ACTIVITY

Instruction:

Explore Plotly's animation features to understand how to create dynamic, interactive visualizations that illustrate changes in data over time or by category.

https://plotly.com/python/animations/



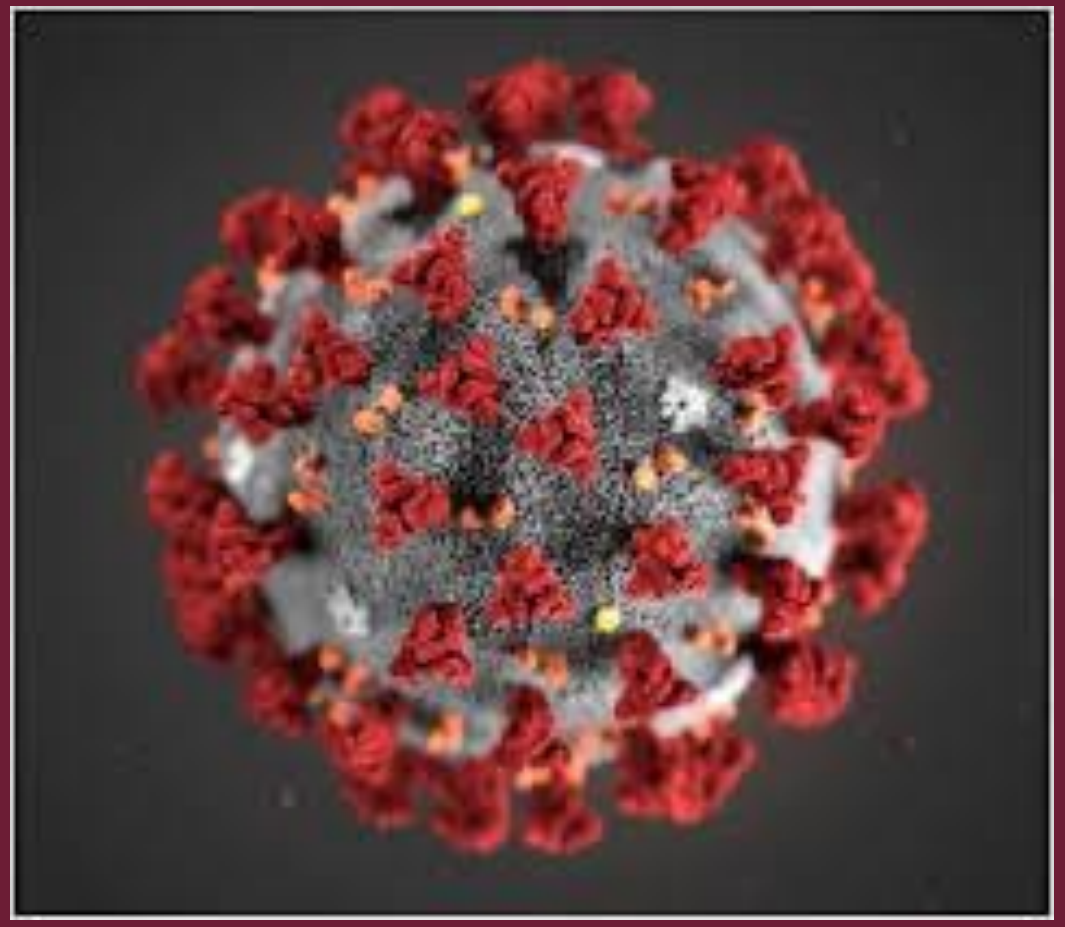


CLASS ACTIVITY

Case Study	Tittle	Key Learnings
1	COVID-19 Time-Series	Animation enhances time-series insights
2	Airbnb Map & Filters	Maps and filters aid spatial data analysis

Case Study 1 – COVID-19 Time-Series

- **Goal:** Visualize COVID-19 cases over time.
- Features:
  i. Animated bar chart
  ii. Hover for country details
  iii. Time slider
- Class Activity:
  i. Explore top 5 countries by daily case trends
  ii. Discuss insights from animation



Code Example – COVID-19 Chart

```
fig = px.line(covid_df, x='date', y='cases', color='country',
              hover_data=['deaths'], animation_frame='date',
              title='COVID-19 Daily Cases')
fig.show()
```

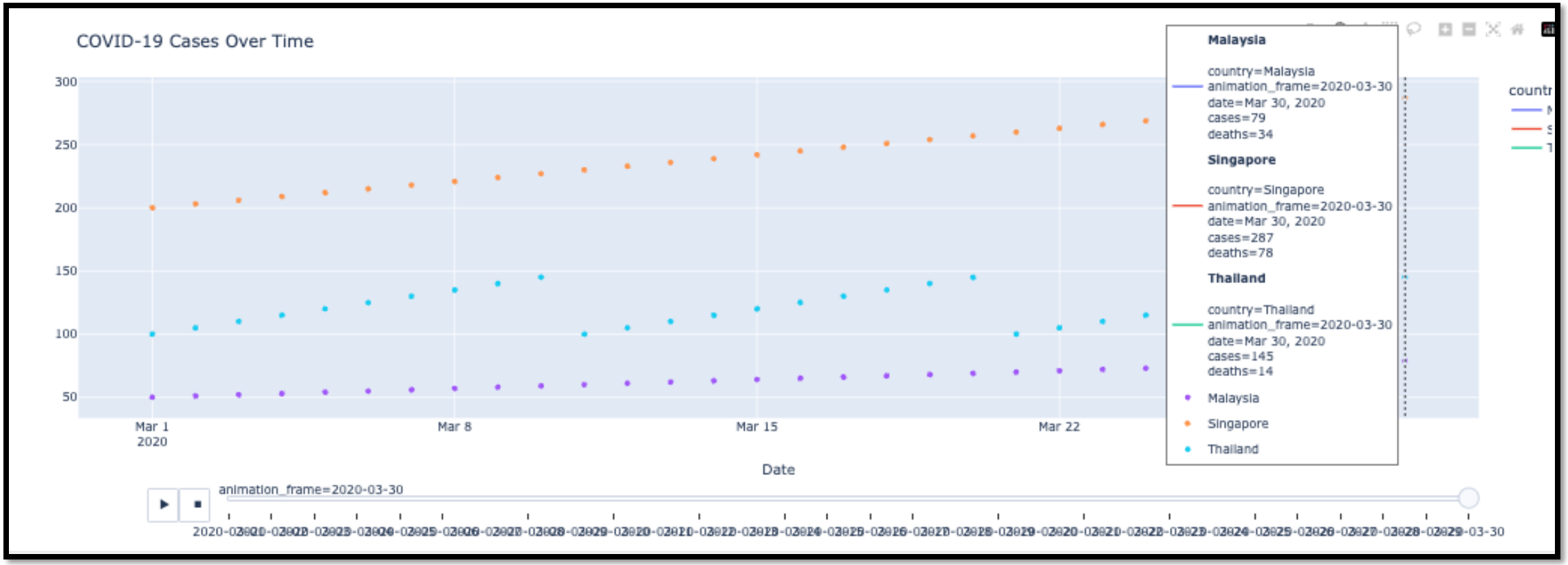
Output:

- Line chart with animation by date
- Hover tooltips for deaths
- Country-based grouping

Data:

- Sample_covid_timeseries.csv

Output Example



Google Colab file:

COVID19_Plotly_Interactive_StudentNotebook..ipynb

Case Study 2 – Airbnb Map & Filters

- Goal: Explore Airbnb listing distribution.
- Features:
  i. Create n=300
  ii. Room type: ‘Entire home/apt', 'Private room', 'Shared room'
  iii. Histogram of room types
  iv. Filter listings by price <=200
  v. Map with hover tooltips
- Class Activity:
  i. Build an interactive map using scatter_mapbox
  ii. Add tooltips and filters to enhance exploration
  iii. Understand how visualizations can support rental decision-making

Code Example – Airbnb Map

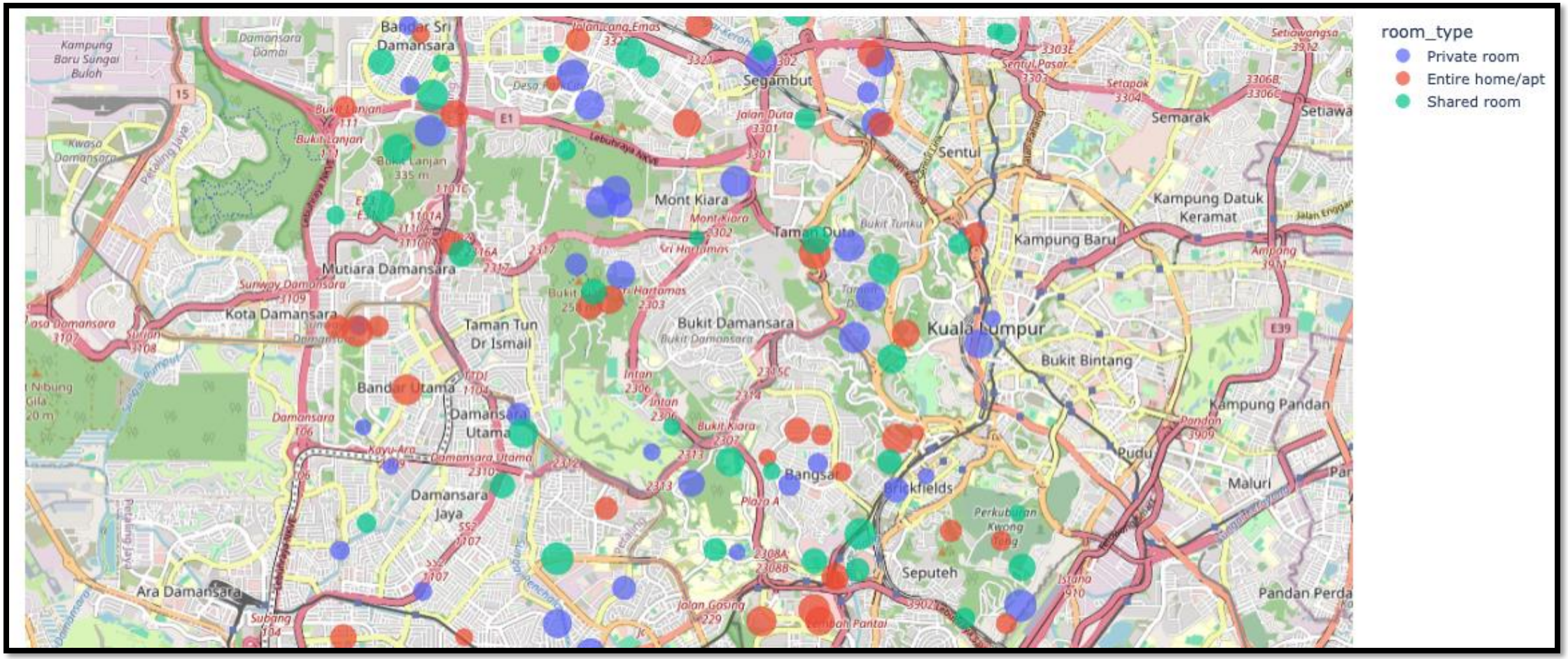
```
# 🗺️ Step 4: Interactive Map with Hover Tooltips
fig_map = px.scatter_mapbox(filtered_df,
                            lat="latitude",
                            lon="longitude",
                            color="room_type",
                            size="price",
                            hover_name="name",
                            hover_data={"price": True, "latitude": False, "longitude": False},
                            zoom=12,
                            height=600,
                            title="Filtered Airbnb Listings (Price ≤ RM200)")

# Mapbox token not needed for open style
fig_map.update_layout(mapbox_style="open-street-map")
fig_map.update_layout(margin={"r":0,"t":40,"l":0,"b":0})
fig_map.show()
```

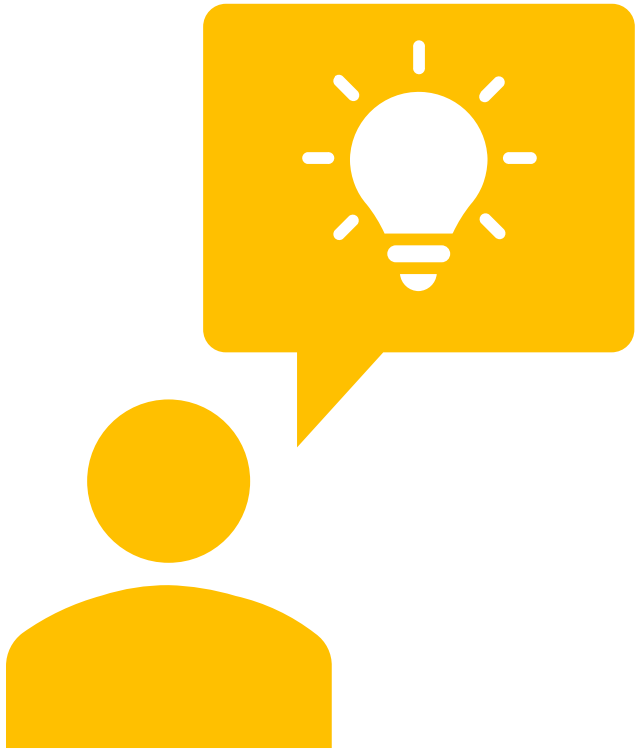
Output:

- Map with colored points
- Zoom, pan, and hover enabled

Output Example



KNOWLEDGE CHECK



THANK YOU



Faculty of Artificial Intelligence (FAI)



http://fai.utm.my



Faculty of Artificial Intelligence, UTM



utmfai